# Naive String-Matching Algorithm

## 1 Idea of the Naive Algorithm

**Core Idea**

The naive string-matching algorithm checks the pattern against the text at **every possible shift**.
At each position, it compares the pattern characters one by one with the corresponding text characters:

$$P[1:m] \stackrel{?}{=} T[s+1:s+m].$$

If all characters match, then that shift is reported as a valid occurrence.

**Why is it called "naive"?**

It is called naive because it tries all possible alignments directly and does **not use any information** learned from previous mismatches.

## 2 How the Algorithm Works

Suppose:

$$T[1:n] = \text{text of length } n, \qquad P[1:m] = \text{pattern of length } m.$$

Then the possible shifts are:

$$s = 0, 1, 2, \ldots, n-m.$$

For each shift $s$, the algorithm checks whether

$$P[1:m] = T[s+1:s+m].$$

If yes, then the pattern occurs at shift $s$.

**Matching Condition**

A shift $s$ is valid if

$$T[s+j] = P[j] \quad \text{for every } 1 \le j \le m.$$

## 3 Pseudocode

**Naive String Matcher**

```
NAIVE-STRING-MATCHER(T, P)
1  n = T.length
2  m = P.length
3  for s = 0 to n - m
4       if P[1 : m] == T[s + 1 : s + m]
5           print "Pattern occurs with shift", s
```

**Explanation of Each Step**

- Line 1: Find the length of the text.
- Line 2: Find the length of the pattern.
- Line 3: Try every possible shift.
- Line 4: Compare the whole pattern with the current text window.
- Line 5: If they are equal, print the shift.

## 4 Template View of the Algorithm

The naive algorithm can be imagined as sliding the pattern like a **template** over the text.

**Text $T$:** a c a a b c

**Pattern $P$:** a a b

First alignment: $s = 0$

At each step, the pattern is shifted one position to the right and compared again.

## 5 Proper Example 1: Pattern Found Once

Let

$$T = \texttt{acaabc}, \qquad P = \texttt{aab}.$$

Here:

$$n = 6, \qquad m = 3.$$

So the possible shifts are:

$$0, 1, 2, 3.$$

### Shift $s = 0$

Compare:

$$T[1:3] = \texttt{aca}, \qquad P = \texttt{aab}.$$

Character comparison:

$$\texttt{a} = \texttt{a}, \qquad \texttt{c} \neq \texttt{a}.$$

So this shift is invalid.

### Shift $s = 1$

Compare:

$$T[2:4] = \texttt{caa}, \qquad P = \texttt{aab}.$$

First character already mismatches:

$$\texttt{c} \neq \texttt{a}.$$

So this shift is invalid.

### Shift $s = 2$

Compare:

$$T[3:5] = \texttt{aab}, \qquad P = \texttt{aab}.$$

All characters match:

$$\texttt{a} = \texttt{a}, \quad \texttt{a} = \texttt{a}, \quad \texttt{b} = \texttt{b}.$$

So the pattern occurs at:

$$\boxed{s = 2}$$

### Shift $s = 3$

Compare:

$$T[4:6] = \texttt{abc}, \qquad P = \texttt{aab}.$$

Second character mismatches:

$$\texttt{a} = \texttt{a}, \qquad \texttt{b} \neq \texttt{a}.$$

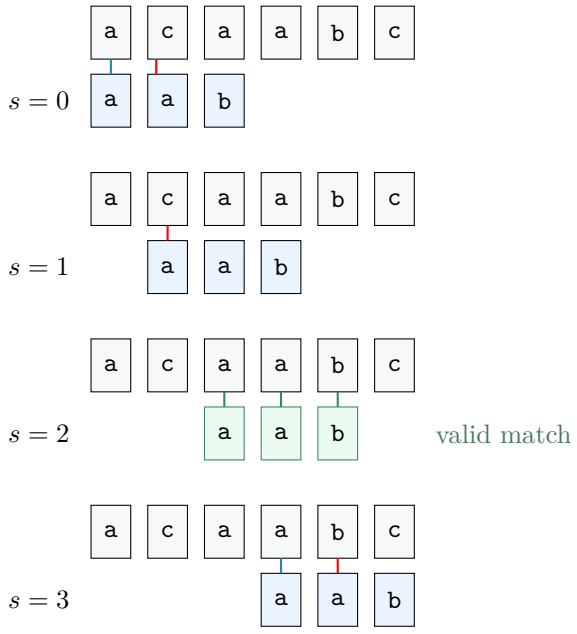
So this shift is invalid.

**Conclusion of Example 1**

The naive algorithm finds exactly one occurrence of the pattern:

$$\boxed{\text{Pattern occurs with shift 2}}$$

### Visual Illustration



## 6 Proper Example 2: Multiple Comparisons

Let

$$T = \texttt{aaaaab}, \qquad P = \texttt{aaab}.$$

Then:

$$n = 6, \qquad m = 4,$$

and possible shifts are:

$$0, 1, 2.$$

### Shift $s = 0$

Compare:

$$\texttt{aaaa} \quad \text{with} \quad \texttt{aaab}$$

First three characters match, but the fourth mismatches:

$$\texttt{a} = \texttt{a}, \quad \texttt{a} = \texttt{a}, \quad \texttt{a} = \texttt{a}, \quad \texttt{a} \neq \texttt{b}.$$

### Shift $s = 1$

Compare:

$$\texttt{aaaa} \quad \text{with} \quad \texttt{aaab}$$

Again, three characters match and the last one mismatches.

### Shift $s = 2$

Compare:

$$\texttt{aaab} \quad \text{with} \quad \texttt{aaab}$$

All characters match, so this is a valid shift.

**Important Lesson**

The naive algorithm repeats many comparisons again and again.
Even after learning that several characters matched before, it does not reuse that information.

## 7 Running Time Analysis

There are exactly

$$n - m + 1$$

possible shifts.

At each shift, in the worst case, the algorithm may compare up to

$$m$$

characters.

Therefore, the worst-case running time is:

$$\boxed{\mathcal{O}((n-m+1)m)}$$

**Worst-Case Time**

$$T(n,m) = \Theta((n-m+1)m)$$

## 8 Worst-Case Example

Consider:

$$T = a^n \quad \text{(a string of } n \text{ a's)}$$

and

$$P = a^m \quad \text{(a string of } m \text{ a's).}$$

For every possible shift:

- all $m$ characters match,
- so each shift requires $m$ comparisons.

Since there are $(n-m+1)$ shifts, total work is:

$$(n-m+1)m$$

Hence the algorithm takes:

$$\boxed{\Theta((n-m+1)m)}$$

If

$$m = \left\lfloor \frac{n}{2} \right\rfloor,$$

then this becomes

$$\Theta(n^2).$$

**Why this is bad**

So in the worst case, the naive algorithm can become quadratic, which is slow for large texts.

## 9 Main Limitation of the Naive Algorithm

**Key Weakness**

The naive algorithm completely ignores useful information from earlier comparisons.

For example, if the pattern is

$$P = \texttt{aaab}$$

and shift $s = 0$ is valid, then some nearby shifts can be ruled out immediately.

But the naive algorithm still checks them one by one.

So it performs unnecessary repeated work.

**Why Better Algorithms Exist**

Algorithms like:

- Rabin–Karp,
- Finite Automaton Matcher,
- Knuth–Morris–Pratt (KMP)

try to avoid this repeated comparison.

## 10 Advantages and Disadvantages

| Feature | Discussion |
|---|---|
| **Simplicity** | Very easy to understand and implement |
| **Preprocessing** | No preprocessing required |
| **Memory usage** | Very small |
| **Efficiency** | Can be slow in the worst case |
| **Reusability of information** | Does not reuse previous matching information |
| **Best use** | Small inputs or when simplicity is more important than speed |

## 11 Exam-Oriented Short Note

**Short Definition**

The naive string-matching algorithm checks the pattern at every possible shift in the text. For each shift, it compares the pattern with the corresponding substring of the text character by character. If all characters match, the shift is reported as a valid occurrence.

**Time Complexity**

$$\boxed{\text{Worst-case time } = \mathcal{O}((n-m+1)m)}$$

$$\boxed{\text{No preprocessing time}}$$

## 12 Final Summary

**One-box Summary**

Try every possible shift $s = 0, 1, \ldots, n - m$

At each shift, compare $P[1 : m]$ with $T[s + 1 : s + m]$

If all characters match, report the shift

Worst-case running time: $\Theta((n - m + 1)m)$

Main drawback: repeated comparisons are not reused